V1 Bar级最小闭环系统设计文档

——面向个人期货量化研究平台的第一代交易状态演化系统

0. 文档定位

本文定义个人期货量化研究平台在数据引擎完成之后的第一个交易系统版本：

> **V1 Bar级最小闭环（Minimum Bar-based Trading Loop）**

V1的目标不是建设一套功能完整的期货回测平台，而是第一次把：

> 市场数据
> → 策略判断
> → 目标仓位
> → 交易订单
> → 模拟成交
> → 实际持仓
> → 账户权益
> → 绩效结果

连接成一个具有明确状态演化过程的完整系统。

其核心设计原则是：

> **架构上闭环，业务语义上极简。**

所谓“架构上闭环”，意味着从V1开始就不能再使用：

```
Signal × Return = Strategy Return
```

这种表达作为正式回测内核。

即使第一版成交模型非常简单，也必须建立：

```
MarketEvent
    ↓
Strategy
    ↓
TargetPosition
    ↓
Order
    ↓
Execution
    ↓
```

```
Fill
    ↓
Account
    ↓
Performance
```

所谓“业务语义上极简”，则意味着V1暂时不追求：

- 完整保证金体系；
- 今仓与昨仓；
- 平今手续费；
- 逐日盯市；
- 限价单；
- 部分成交；
- 挂单；
- Tick撮合；
- 订单队列；
- 实盘接口；
- 多账户；
- 复杂风险预算。

这些能力以后可以增加，但它们不应阻碍第一条完整交易生命周期尽快跑通。

一、V1要回答的核心研究问题

1.1 从“价格规律”升级到“可交易策略”

在V0研究原型阶段，可以使用向量化方式快速验证：

```
signal = np.sign(close.pct_change(20))
strategy_return = signal.shift(1) * close.pct_change()
```

这种研究能够回答：

> 过去价格变化与未来收益之间是否存在值得继续研究的关系？

但它不能回答：

> 一个研究者在历史时间点真实观察到这个信号以后，能否通过一系列具体交易动作获得这样的收益？

V1的研究问题因此发生第一次重要升级：

> **一个已经定义好的简单信号，在明确的信息时点、成交时点、物理合约和账户状态约束下，能否形成一条自洽的策略权益曲线？**

V1并不试图证明策略已经足够真实。

它只是要求：

> **收益必须由交易过程产生，而不能由研究者直接计算出来。**

这是从“策略研究脚本”进入“回测系统”的分界线。

二、V1的总体范围

2.1 V1支持什么

第一版建议只支持：

市场数据

- 日频Bar；
- 或后续增加1h等低频Bar；
- 单一品种起步；
- 当前主力物理合约；
- 复权连续价格用于研究信号；
- 原始物理合约价格用于成交。

策略

第一条主线策略建议：

> **20 Bar时间序列趋势策略**

即：

```
20 Bar累计收益 > 0 → TargetPosition = +1
20 Bar累计收益 < 0 → TargetPosition = -1
否则 → TargetPosition = 0
```

第一版不进行复杂波动率调整。

仓位

只支持：

```
-1手
0手
+1手
```

或者通过配置指定固定数量：

```
-quantity
0
+quantity
```

订单

只支持：

> Market Order

即市价交易意图。

成交

采用统一简化假设：

> 当前Bar收盘后产生信号；
> 下一根Bar开盘价成交；
> 订单全部成交。

可以额外加入：

```
execution_price
=
next_bar_open
+
固定滑点
```

账户

维护：

- 初始资金；
- 现金；
- 实际持仓；
- 持仓成本；
- 已实现盈亏；
- 未实现盈亏；
- 手续费；
- 总权益。

绩效

至少输出：

- 总收益率；
- 年化收益率；
- 年化波动率；

- Sharpe；
- 最大回撤；
- 交易次数；
- 总手续费；
- 净收益；
- 每日/每Bar权益曲线。

三、V1明确不做什么

V1最大的开发风险不是功能不足，而是开发范围不断扩张。

因此应明确以下内容全部属于后续版本。

3.1 暂不实现完整保证金体系

V1可以：

- 完全忽略保证金约束；

或者：

- 使用一个非常简单的名义资金检查。

但不要立即实现：

- 初始保证金；
- 维持保证金；
- 动态保证金；
- 交易所保证金；
- 经纪商保证金加成；
- 风险度；
- 强制平仓。

这些进入V2。

3.2 暂不实现今仓与昨仓

第一版：

```
Position = signed quantity
```

即可。

暂时不区分：

```
LongToday
LongYesterday
ShortToday
ShortYesterday
```

这些进入期货账户语义阶段。

3.3 暂不实现真实逐日盯市

V1可以按照普通mark-to-market逻辑维护：

```
Equity
=
Cash
+
Unrealized PnL
```

而不立即模拟中国期货结算制度中：

- 结算价；
- 每日结算；
- 浮盈转现金；
- 持仓成本重置；
- 保证金重新计算。

这些进入V2。

3.4 暂不支持限价单

只有：

```
MarketOrder
```

暂不实现：

- LimitOrder；
- StopOrder；
- StopLimitOrder；
- IOC；
- FOK；
- 撤单。

3.5 暂不支持部分成交

V1统一假设：

```
Order Quantity = Fill Quantity
```

暂时没有：

```
PENDING
PARTIALLY_FILLED
CANCELLED
REJECTED
```

复杂订单生命周期进入后续版本。

3.6 暂不使用Tick撮合

现有数据引擎已经拥有 `SimulationFeed`，其职责是严格按照时间顺序输出主力合约的市场快照，换月期还能够同时输出新旧两个物理合约。

其底层甚至已经利用时间堆和零阶保持合并新旧合约Tick，并处理同一时间戳多个Tick的情况。

但V1**不要使用它**。

原因不是它不好，而是：

> Tick数据会立即把学习者带入成交顺序、盘口、微观流动性和事件同步问题。

这些都不是V1最需要学习的问题。

V1的第一任务是理解：

> **一次策略判断如何经过Target → Order → Fill → Account真正改变账户状态。**

因此现有 `SimulationFeed` 应被保留，作为未来V5的天然接入点，而不是现在立即消费。

四、V1与现有数据引擎的边界

这是本设计中非常重要的一部分。

V1不应该重新构造：

- 交易日历；
- Bar聚合；

- 主力合约识别；
- 连续合约；
- 复权；
- 真实收益；
- 文件读取；
- 清洗。

这些全部继续由现有数据引擎负责。

现有 `DataFacade` 已经提供：

```
get_historical_vector(...)
```

用于获取复权连续主力Bar，以及：

```
get_snapshot_stream(...)
```

用于获取逐笔市场快照。其历史行情接口支持1min、5min、15min、30min、1h、2h、4h和1d等频率。

对于V1：

> **只使用Bar历史接口。**

五、V1最重要的数据语义：研究价格与成交价格分离

这是V1必须从第一天就守住的架构不变量。

现有 `ResearchFeed` 输出的Bar中已经包含：

- `Dominant_Contract`
- `Price_Gap`
- `Adj_Open`
- `Adj_High`
- `Adj_Low`
- `Adj_Close`
- `Raw_Open`
- `Raw_High`
- `Raw_Low`
- `Raw_Close`
- `True_Return`
- `is_roll`

其中：

- `Adj_*` 是复权后的连续价格；
- `Raw_*` 是当时主力物理合约的真实价格；
- `Dominant_Contract` 指明实际物理合约；

- `True_Return` 已对换月收益进行专门处理。

因此V1应采用非常明确的原则：

5.1 Strategy使用什么价格？

研究趋势时使用：

```
Adj_Close
```

原因是：

> Strategy研究的是连续市场价格运动。

例如：

```
momentum = Adj_Close[t] / Adj_Close[t-20] - 1
```

5.2 Execution使用什么价格？

实际成交必须使用：

```
Raw_Open
```

并且订单对应：

```
Dominant_Contract
```

不能使用：

```
Adj_Open
```

成交。

因为复权价格并不是交易所真实存在的价格。

5.3 为什么必须在V1就区分？

因为如果V1采用：

```
连续合约价格
=
账户成交价格
```

那么未来加入物理合约换月时，会发生核心语义推翻。

正确设计应该从一开始就承认：

```
Research Asset
≠
Tradable Contract
```

例如：

```
Research Symbol: RB.CONTINUOUS

当前实际交易物理合约:
RB2510
```

策略判断的是：

```
RB趋势向上
```

账户真正持有的是：

```
RB2510 +1手
```

这一区分现在几乎不会增加多少开发成本，却可以避免未来非常昂贵的重构。

六、V1核心数据流

V1正式的数据流定义为：

```
DataFacade / ResearchFeed
        ↓
BarMarketFeed
        ↓
BarMarketEvent
        ↓
BacktestEngine
        ↓
Strategy
```

```
        ↓
TargetPosition
        ↓
OrderManager
        ↓
Order
        ↓
ExecutionModel
        ↓
Fill
        ↓
Account
        ↓
AccountSnapshot
        ↓
PerformanceRecorder
```

这里应该特别注意：

> 不需要为了“事件驱动”而立刻设计复杂EventBus。

V1可以使用非常简单的顺序循环。

“事件驱动”的本质不是有没有EventBus，而是：

> **系统状态变化由明确事件触发。**

七、核心对象一：BarMarketEvent

7.1 职责

`BarMarketEvent` 表示：

> 一根已经完整结束、现在正式对策略可见的市场Bar。

建议字段：

```
@dataclass(frozen=True)
class BarMarketEvent:
    product: str
    contract: str

    timestamp: pd.Timestamp
    trading_date: date

    adj_open: float
    adj_high: float
```

```
    adj_low: float
    adj_close: float

    raw_open: float
    raw_high: float
    raw_low: float
    raw_close: float

    is_roll: bool
```

V1甚至可以暂时不加入：

```
volume
open_interest
price_gap
```

除非策略真的需要。

7.2 为什么不直接把DataFrame传给Strategy?

可以，但不推荐。

因为：

```
DataFrame row
```

是数据存储表示；

而：

```
BarMarketEvent
```

是交易系统语义。

将两者分开以后，将来：

```
ResearchFeed
```

可以被别的数据源替换，而策略不需要理解DataFrame列名。

八、核心对象二：Strategy

8.1 Strategy职责边界

Strategy只回答：

> 根据目前已经合法可见的市场历史，我希望持有什么方向？

Strategy不能：

- 修改账户；
- 创建Fill；
- 计算手续费；
- 决定成交价格；
- 修改现金；
- 直接计算最终策略收益。

8.2 第一版接口

可以设计：

```
class Strategy(ABC):

    @abstractmethod
    def on_bar(
        self,
        event: BarMarketEvent,
        context: StrategyContext,
    ) -> TargetPosition | None:
        ...
```

其中：

```
event
```

是最新完成的Bar；

```
context
```

提供合法历史数据以及当前账户只读信息。

九、StrategyContext：控制信息边界

这是一个值得V1就建立的小型对象。

9.1 为什么需要Context?

如果直接把整个未来DataFrame传给Strategy：

```
strategy.run(all_data)
```

那么策略理论上能够访问：

```
t+1
t+2
...
```

未来数据。

即使当前代码没有主动访问，也缺少制度性保护。

因此建议：

```
class StrategyContext:
    history: HistoryView
    account: ReadOnlyAccountView
```

Strategy只能访问：

```
截至当前event的数据
```

9.2 V1不用做复杂历史数据库

一个简单：

```
deque
```

或者：

```
list[BarMarketEvent]
```

即可。

例如：

```
context.history.last(20)
```

十、核心对象三：TargetPosition

10.1 为什么需要TargetPosition?

这是整个架构最关键的概念之一。

Strategy不应该说：

> 买1手。

Strategy应该说：

> 我希望最终持有+1手。

因此：

```
@dataclass(frozen=True)
class TargetPosition:
    product: str
    target_quantity: int
    generated_at: pd.Timestamp
```

例如：

```
当前实际仓位 = -1
目标仓位 = +1
```

那么系统下一层才计算：

```
需要买入2手
```

10.2 Signal与TargetPosition是否都需要?

V1可以不单独建立Signal类。

因为：

```
momentum > 0
```

属于Strategy内部逻辑。

为了防止过度工程，可以直接：

```
Strategy → TargetPosition
```

以后因子与信号框架成熟后，再明确：

```
Factor
→ Signal
→ TargetPosition
```

十一、核心对象四：Order

11.1 Order是什么?

Order表示：

> 系统准备向市场表达的交易意图。

它不同于TargetPosition。

例如：

```
ActualPosition = -1
TargetPosition = +1
```

需要：

```
Order = BUY 2
```

11.2 V1 Order字段

```
@dataclass(frozen=True)
class Order:
```

```
    order_id: str

    product: str
    contract: str

    side: Literal["BUY", "SELL"]
    quantity: int

    created_at: pd.Timestamp
```

暂时不需要：

- order_type；
- limit_price；
- stop_price；
- TIF；
- status；
- exchange_order_id。

因为V1只支持：

```
市价单
+
下一Bar全部成交
```

十二、核心模块：OrderManager

12.1 为什么TargetPosition不能直接交给Execution?

因为需要一个模块计算：

```
TargetPosition
-
ActualPosition
=
Required Trade
```

这个职责可以放在：

```
OrderManager
```

中。

例如：

```
delta = target.target_quantity - account.position(product)
```

若：

```
delta > 0 → BUY
delta < 0 → SELL
delta = 0 → no order
```

12.2 物理合约解析

OrderManager还应负责确定：

```
这一订单实际交易哪个合约？
```

V1可以直接使用当前Bar提供的：

```
Dominant_Contract
```

即：

```
TargetPosition(product="RB", +1)

↓

Order(
    contract="RB2510",
    side="BUY",
    quantity=1
)
```

十三、核心对象五：PendingOrder

因为V1采用：

> 当前Bar收盘产生信号，下一Bar开盘成交。

因此Order生成以后不能立即Fill。

系统必须至少拥有：

```
pending_orders
```

队列。

这是信息时点正确性的关键。

流程是：

```
Bar t Close
↓
Strategy
↓
Target
↓
Order
↓
pending_orders

--------------------------------

Bar t+1 arrives
↓
使用 Bar t+1 Raw_Open
↓
Fill
```

这一步看似很小，却真正阻断了：

> 用收盘后信息按同一收盘价成交

这一类Look-ahead bias。

十四、核心模块：ExecutionModel

14.1 职责

ExecutionModel只回答：

> 给定一个已经存在的Order和下一根市场Bar，它如何成交？

14.2 V1执行模型

第一版：

```
class NextBarOpenExecutionModel:

    def execute(
        self,
        order: Order,
        bar: BarMarketEvent,
    ) -> Fill:
        ...
```

成交价格：

```
BUY:

fill_price
=
Raw_Open
+
slippage

SELL:

fill_price
=
Raw_Open
-
slippage
```

14.3 SlippageModel

不要把滑点直接写死在Execution内部。

可以设计极简：

```
class FixedSlippageModel:

    def calculate(
        self,
        side,
        raw_open,
    ) -> float:
        ...
```

第一版甚至可以：

```
slippage = 0
```

但接口应存在。

这属于：

> **提前留下替换点，而不提前实现复杂业务。**

十五、核心对象六：Fill

15.1 Fill是什么？

Fill代表：

> 市场已经确认发生的交易事实。

只有Fill可以改变实际账户持仓。

15.2 字段

```
@dataclass(frozen=True)
class Fill:
    fill_id: str
    order_id: str

    product: str
    contract: str

    side: Literal["BUY", "SELL"]
    quantity: int

    price: float

    timestamp: pd.Timestamp

    commission: float
```

十六、CommissionModel

V1手续费不应追求真实中国期货全部规则。

只需要支持一种简化模型，例如：

```
commission
=
quantity × fixed_fee_per_contract
```

或者：

```
commission
=
notional × rate
```

接口：

```
class CommissionModel:

    def calculate(
        self,
        contract: str,
        side: str,
        quantity: int,
        price: float,
    ) -> float:
        ...
```

目前不要实现：

- 开仓手续费；
- 平仓手续费；
- 平今手续费；
- 平昨手续费；
- 品种历史手续费变化。

这些进入V2。

十七、核心模块：Account

这是V1真正开始进入“交易状态演化”的地方。

17.1 Account必须是唯一权威来源

所有：

- 实际仓位；
- 持仓成本；
- 现金；
- 已实现盈亏；

- 未实现盈亏；
- 手续费；
- Equity；

全部由Account维护。

Strategy不能自己维护第二套账户。

Performance也不能重新独立计算一套账户。

十八、V1 Position模型

第一版可以设计：

```
@dataclass
class Position:
    product: str
    contract: str

    quantity: int
    average_price: float
```

其中：

```
quantity > 0 → Long
quantity < 0 → Short
quantity = 0 → Flat
```

暂不区分：

- long today；
- long yesterday；
- short today；
- short yesterday。

十九、Account.apply_fill()

这是系统最核心的方法之一。

原则：

> **只有 `apply_fill()` 能改变Position。**

例如：

```
account.apply_fill(fill)
```

负责：

1. 读取原持仓；
2. 判断增仓、减仓、平仓或反向；
3. 更新quantity；
4. 更新average_price；
5. 计算realized PnL；
6. 扣commission；
7. 更新cash。

二十、反向交易必须在V1就正确处理

例如：

```
当前：
Short 1 @ 3500

收到：
BUY 2 @ 3510
```

结果不是简单：

```
Long 1 @ 某加权均价
```

而应该理解为：

```
BUY 1
用于平掉 Short 1

+

BUY 1
形成新的 Long 1
```

因此：

```
原Short产生Realized PnL

新的Long成本价 = 3510
```

这是V1账户模块最值得亲自实现和理解的部分之一。

因为它第一次迫使开发者理解：

> 交易并不是简单改变一个position整数；成交改变的是一个具有历史成本基础的资产状态。

二十一、Mark-to-Market

每根Bar结束后：

```
account.mark_to_market(bar)
```

使用：

```
当前物理合约 Raw_Close
```

计算：

```
Unrealized PnL
```

例如：

Long

```
(Current Raw Close - Average Price)
× Quantity
× Contract Multiplier
```

Short

```
(Average Price - Current Raw Close)
× abs(Quantity)
× Contract Multiplier
```

二十二、合约乘数必须在V1进入

虽然保证金可以推迟，但：

> **Contract Multiplier不能推迟。**

否则账户盈亏不是真实期货盈亏。

例如：

```
RB价格上涨10元/吨
```

并不意味着：

```
+10元
```

而需要：

```
PnL
=
10
×
合约乘数
×
手数
```

因此Account或者独立ContractSpecificationService必须能够获得：

```
contract_multiplier
```

这部分应直接复用现有品种元数据体系。

二十三、Equity定义

V1可以使用简单账户恒等式：

```
Equity
=
Cash
+
Unrealized PnL
```

其中Cash包含：

```
Initial Cash
+
Realized PnL
-
Commission
```

因此：

```
Equity
=
Initial Cash
+
Realized PnL
+
Unrealized PnL
-
Commission
```

第一版暂不处理：

```
Margin
```

单独占用。

二十四、AccountSnapshot

不要让Performance直接读取Account内部可变对象。

每根Bar结束以后生成：

```
@dataclass(frozen=True)
class AccountSnapshot:
    timestamp: pd.Timestamp

    cash: float

    realized_pnl: float
    unrealized_pnl: float

    commission: float

    equity: float

    positions: dict
```

Performance只记录：

```
AccountSnapshot
```

这样可以保证历史状态不会被未来修改。

二十五、PerformanceRecorder

V1不要开发复杂绩效系统。

其职责只有：

```
保存：
AccountSnapshot
Fill
Order
TargetPosition
```

最终生成：

```
equity_curve
trade_log
order_log
position_history
```

绩效指标可以大量复用成熟库。

第一版只需确保：

> **所有绩效结果都能追溯到账户状态和真实Fill。**

二十六、BacktestEngine：整个系统的调度核心

BacktestEngine不应该包含所有业务逻辑。

它的职责是：

> 确保模块按照正确时间顺序被调用。

二十七、V1的关键事件顺序

这是整个系统设计最重要的一段。

对于每一个新的Bar：

```
① 新Bar进入
```

```
↓

② 执行上一根Bar结束后形成的Pending Orders
   成交价格 = 当前Bar Raw_Open

↓

③ Fill进入Account
   更新实际Position和Cash

↓

④ 当前Bar完整结束
   Account按照当前Raw_Close mark-to-market

↓

⑤ BarMarketEvent正式对Strategy可见

↓

⑥ Strategy利用截至当前Bar的数据计算TargetPosition

↓

⑦ OrderManager比较:
   TargetPosition
   vs
   ActualPosition

↓

⑧ 若不同，生成Order

↓

⑨ Order进入Pending队列

↓

⑩ Account生成当前Bar结束时AccountSnapshot

↓

⑪ PerformanceRecorder记录
```

这一顺序非常重要。

二十八、为什么不能先Strategy再用当前Open成交

假设当前Bar为：

```
2026-01-10
```

其：

```
Open = 3500
Close = 3550
```

如果Strategy使用：

```
Close = 3550
```

计算趋势信号以后，又按：

```
Open = 3500
```

成交：

这显然使用了未来信息。

因此：

> 当前Bar结束后生成的Order，必须等下一根Bar才能成交。

V1通过：

```
pending_orders
```

这一极其简单的结构，就可以从架构上消除这一类错误。

二十九、BacktestEngine伪代码

推荐第一版保持极其直白：

```
class BacktestEngine:

    def run(self):
```

```
        for bar in self.market_feed:

            # 1. 执行上一期订单
            fills = self.execution_model.execute_pending_orders(
                pending_orders=self.pending_orders,
                bar=bar,
            )

            # 2. 成交更新账户
            for fill in fills:
                self.account.apply_fill(fill)
                self.performance.record_fill(fill)

            self.pending_orders.clear()

            # 3. 当前Bar收盘盯市
            self.account.mark_to_market(bar)

            # 4. 更新历史视图
            self.strategy_context.on_bar(bar)

            # 5. Strategy产生目标仓位
            target = self.strategy.on_bar(
                bar,
                self.strategy_context,
            )

            if target is not None:

                self.performance.record_target(target)

                # 6. Target -> Order
                order = self.order_manager.create_order(
                    target=target,
                    account=self.account,
                    market=bar,
                )

                if order is not None:
                    self.pending_orders.append(order)
                    self.performance.record_order(order)

            # 7. 记录账户状态
            snapshot = self.account.snapshot(
                timestamp=bar.timestamp,
            )

            self.performance.record_snapshot(snapshot)
```

V1完全不需要：

```
异步框架
EventBus
消息队列
协程
多线程
Actor Model
```

这些都会掩盖真正需要学习的交易状态。

三十、BarMarketFeed设计

建议在现有DataFacade与BacktestEngine之间增加一个非常薄的适配层：

```
DataFacade
↓
BarMarketFeed
↓
BarMarketEvent
```

30.1 为什么增加这一层？

现有 `ResearchFeed` 返回的是：

```
pd.DataFrame
```

这非常适合研究。

但BacktestEngine最好消费：

```
Iterator[BarMarketEvent]
```

因此：

```
class BarMarketFeed:

    def stream(
        self,
        product,
        start_date,
        end_date,
        freq,
```

```
    ) -> Iterator[BarMarketEvent]:
        ...
```

内部：

```
df = facade.get_historical_vector(...)
```

然后逐行转换成Event。

30.2 现有数据引擎为什么已经足以支持这一设计？

现有 `ResearchFeed` 本身已经完成：

1. 查询主力物理合约映射；
2. 收集对应物理合约行情；
3. 日频与日内频率分别处理；
4. 日内数据清洗；
5. Bar聚合；
6. 连续价格复权；
7. True Return计算。

其Bar聚合器还明确遵循 session-aligned windowing，禁止K线跨越不同交易小节。

因此V1没有理由重新建设市场数据时间轴。

三十一、V1主线策略：20 Bar趋势

可以建立：

```
class TimeSeriesMomentumStrategy(Strategy):
```

配置：

```
@dataclass(frozen=True)
class MomentumConfig:
    lookback: int = 20
    quantity: int = 1
```

逻辑：

```
if len(history) < lookback + 1:
    return None
```

```
past = history[-lookback - 1].adj_close
current = history[-1].adj_close

momentum = current / past - 1

if momentum > 0:
    target = +quantity
elif momentum < 0:
    target = -quantity
else:
    target = 0
```

三十二、为什么V1选择如此简单的策略

因为V1现在测试的不是：

> 能否发现最好的Alpha。

而是：

> 整个系统是否能够把一个清晰的研究假说正确转化为账户状态。

如果策略本身包含：

- 十几个指标；
- 动态参数；
- 机器学习；
- 多周期信号；
- 复杂过滤条件；

那么系统出现异常时，很难判断究竟来自：

- 策略；
- 数据；
- 执行；
- 账户；
- 时间逻辑。

因此：

> **复杂策略会污染V1系统开发的诊断环境。**

三十三、V1对换月如何处理

这是一个需要特别谨慎的问题。

从架构原则上：

> V1不能把连续合约直接当账户资产。

但V1也不应该立即建设完整换月执行系统。

因此建议采用：

> **最小物理合约切换模型。**

33.1 当Dominant_Contract不变

正常：

```
TargetPosition(RB) = +1

Account:
RB2510 +1
```

无需额外操作。

33.2 当Dominant_Contract变化

例如：

```
昨日：
Dominant = RB2510

今日：
Dominant = RB2601
```

如果账户仍持有：

```
RB2510 +1
```

而Target仍然：

```
RB +1
```

V1需要产生：

```
SELL RB2510 1
BUY  RB2601 1
```

即：

> 最简单的机械换月。

第一版可以假设两笔交易都在新的主力切换后的下一Bar开盘成交。

暂时忽略：

- 换月提前期优化；
- 双合约过渡；
- 流动性冲击；
- 分批换月；
- 新旧合约不同滑点模型。

33.3 为什么V1仍然要有最小换月？

因为否则：

```
TargetPosition(product)
```

与：

```
ActualPosition(contract)
```

之间的映射会缺失。

而这种缺失以后不是局部功能增加，而是核心资产身份重构。

因此：

> 完整换月可以推迟，
> 但“研究品种不等于交易合约”这一语义不能推迟。

三十四、V1的目录结构建议

可以在现有：

```
data/
```

之外增加：

```
strategy/
    base.py
    context.py
    momentum.py

portfolio/
    models.py
    target_manager.py

execution/
    models.py
    execution_model.py
    commission.py
    slippage.py

account/
    models.py
    account.py

backtest/
    engine.py
    market_feed.py
    config.py

analytics/
    recorder.py
    performance.py

tests/
    strategy/
    execution/
    account/
    backtest/
```

三十五、是否需要events目录？

V1可以暂时不创建：

```
events/
```

因为：

```
BarMarketEvent
TargetPosition
Order
Fill
AccountSnapshot
```

分别放在各自领域模块更清楚。

以后事件类型明显增多时，再抽象统一：

```
events/
```

这体现：

延迟复杂抽象。

三十六、对象依赖方向

推荐：

```
data
  ↓
backtest.market_feed
  ↓
strategy
  ↓
portfolio
  ↓
execution
  ↓
account
  ↓
analytics
```

但BacktestEngine作为编排器可以依赖所有模块。

业务模块之间尽量不要形成反向依赖。

例如：

```
Strategy
```

不能import：

```
ExecutionModel
```

Account也不应该知道Strategy是什么。

三十七、V1配置

建议统一：

```
@dataclass(frozen=True)
class BacktestConfig:
    product: str
    start_date: date
    end_date: date

    freq: str = "1d"

    initial_cash: float = 1_000_000

    commission_per_contract: float = 0.0
    slippage_per_contract: float = 0.0
```

Strategy参数单独：

```
MomentumConfig
```

不要把所有参数放入一个MegaConfig。

三十八、一次完整交易示例

这是V1最重要的认知案例。

假设：

```
Initial Cash = 1,000,000
Contract Multiplier = 10
Commission = 2元/手
Slippage = 1元价格单位
```

Bar 20结束：

```
Adj_Close趋势 > 0
```

Strategy：

```
TargetPosition = +1
```

Account：

```
ActualPosition = 0
```

OrderManager：

```
BUY 1 RB2510
```

订单进入：

```
PendingOrders
```

下一Bar：

```
Raw_Open = 3500
```

Execution：

```
Fill Price = 3501
```

Fill：

```
BUY 1 RB2510 @ 3501
Commission = 2
```

Account：

```
Position:
RB2510 +1 @ 3501

Cash:
1,000,000 - 2
```

Bar收盘：

```
Raw_Close = 3520
```

Unrealized PnL：

```
(3520 - 3501)
× 10
× 1
=
190
```

Equity：

```
999,998
+
190
=
1,000,188
```

Performance记录：

```
Target
Order
Fill
Position
AccountSnapshot
```

整个收益由交易状态自然产生。

这才是V1的核心成果。

三十九、反向持仓示例

假设下一次信号：

```
TargetPosition = -1
```

当前：

```
ActualPosition = +1
```

因此：

```
delta
=
-1 - (+1)
=
-2
```

Order：

```
SELL 2
```

下一Bar成交以后：

第一手：

```
关闭Long 1
```

产生Realized PnL。

第二手：

```
建立Short 1
```

形成新的Average Price。

最终：

```
ActualPosition = -1
```

这一个案例应成为Account模块第一批黄金路径测试之一。

四十、V1最重要的不变量

40.1 Position不变量

```
没有Fill
→
Position绝不变化
```

40.2 时间不变量

```
Fill.timestamp
>
Order.created_at
```

在V1中原则上是下一Bar。

40.3 资产身份不变量

```
Account Position
必须是Physical Contract
```

不能是：

```
RB.CONTINUOUS
```

40.4 成交价格不变量

Fill必须来自：

```
Raw Price
```

不能来自：

```
Adjusted Price
```

40.5 策略信息不变量

Strategy在Bar t结束时只能访问：

```
⩽ t
```

的数据。

40.6 账户权威不变量

最终：

```
Equity
PnL
Position
```

只能由Account确定。

四十一、测试体系

V1测试的重要性不亚于实现本身。

四十二、第一层：纯模型测试

测试：

```
TargetPosition
Order
Fill
Position
```

对象构造和基本字段约束。

四十三、第二层：OrderManager测试

场景：

Case 1

```
Current = 0
Target = +1

→ BUY 1
```

Case 2

```
Current = +1
Target = +1
```

```
→ None
```

Case 3

```
Current = +1
Target = -1

→ SELL 2
```

四十四、第三层：Execution测试

测试：

```
BUY
Raw_Open = 3500
Slippage = 1

→ Fill = 3501
```

以及：

```
SELL

→ Fill = 3499
```

四十五、第四层：Account黄金路径

至少建立三个手工可计算场景。

Case A：开多 → 平多

```
BUY 1 @ 3500
SELL 1 @ 3520
```

验证：

- Position；
- Average Price；
- Realized PnL；
- Commission；

- Cash；
- Equity。

Case B：开空 → 平空

```
SELL 1 @ 3500
BUY 1 @ 3480
```

Case C：多头直接反空

```
Long 1
↓
SELL 2
↓
Short 1
```

这是最重要的状态转换测试。

四十六、第五层：时间因果测试

专门构造：

```
Bar t Open = 100
Bar t Close = 110
```

信号只有在看到：

```
Close = 110
```

之后才出现。

测试必须保证：

> Fill绝不能发生在Bar t Open=100。

只能发生：

```
Bar t+1 Open
```

这个测试专门保护：

look-ahead bias。

四十七、第六层：端到端黄金测试

构造10—30根人工Bar：

```
先上涨
→ 趋势策略开多

随后下跌
→ 反向做空
```

然后手工计算：

- 每次Target；
- 每次Order；
- 每次Fill；
- Position；
- PnL；
- Equity。

整个BacktestEngine输出必须与手工结果完全一致。

这条测试应成为未来V2、V3重构之后仍然继续运行的核心回归测试。

四十八、V1完成标准

V1不能以：

代码都写完了

作为完成标准。

必须同时满足以下条件。

48.1 能够运行一个完整趋势策略

输入：

```
product
start_date
```

```
end_date
lookback
```

系统可以完整跑完。

48.2 每一笔交易可追踪

对于任何一个Fill，都可以回答：

哪个Bar产生了什么Target?

↓

为什么生成这个Order?

↓

Order什么时候成交?

↓

成交如何改变Position?

↓

最终如何改变Equity?

48.3 没有Signal直接乘收益率

正式回测结果必须完全由：

```
Fill
+
Account
```

产生。

48.4 使用真实物理合约成交价格

即：

```
Research:
Adj Price
```

```
Execution:
Raw Physical Contract Price
```

48.5 时间因果正确

当前Bar产生的信息不能用当前Bar之前的价格成交。

48.6 核心黄金路径测试全部通过

尤其是：

- 开多平多；
- 开空平空；
- 多转空；
- 空转多；
- 下一Bar成交；
- 主力物理合约变化。

四十九、V1完成后第一件事不是开发V2

这是“研究可信度驱动系统演化”最重要的课程原则。

V1完成以后：

> **立即暂停系统开发。**

不要马上开始：

- 保证金；
- 今昨仓；
- Tick；
- 复杂撮合；
- 分布式。

而应该真正使用V1做一轮策略研究。

例如系统研究：

> 20日、60日、120日时间序列趋势。

研究：

- 不同品种；

- 不同频率；
- 不同lookback；
- 有无手续费；
- 不同滑点；
- 不同换月时期。

五十、让策略研究暴露V1真正的不足

实际研究以后可能发现：

问题A

策略理论上频繁开仓，但账户根本没有资金约束。

↓

真实研究需求：

> 加入保证金。

问题B

手续费结果与实际期货交易差异巨大。

↓

需求：

> 开仓/平仓/平今规则。

问题C

主力切换时期收益异常。

↓

需求：

> 完善换月执行。

问题D

多个品种同时交易后，风险差异太大。

↓

需求：

风险标准化与Portfolio。

问题E

日内策略结果对Bar内部价格路径极度敏感。

↓

需求：

Tick SimulationFeed。

这时系统进入下一阶段，是因为：

研究结论已经被当前系统假设限制。

而不是因为：

“开发计划表规定下一步该开发V2。”

五十一、V1之后的架构演化预留

如果上述架构正确，V2增加完整期货账户时：

```
Strategy
TargetPosition
Order
Fill
```

基本不需要改变。

主要扩展：

```
Account
PositionBook
CommissionModel
MarginModel
Settlement
```

以后接入Tick时：

```
Strategy
TargetPosition
Account
Performance
```

仍可基本保留。

主要替换：

```
BarMarketFeed
→ SimulationFeed

NextBarOpenExecution
→ TickExecutionModel
```

现有数据引擎已经拥有 `ContractTick` 和 `MarketSnapshot` 数据模型，其中快照能够同时携带主力合约和换月期旧主力合约，并包含涨跌停状态，为这种未来替换提供了天然基础。

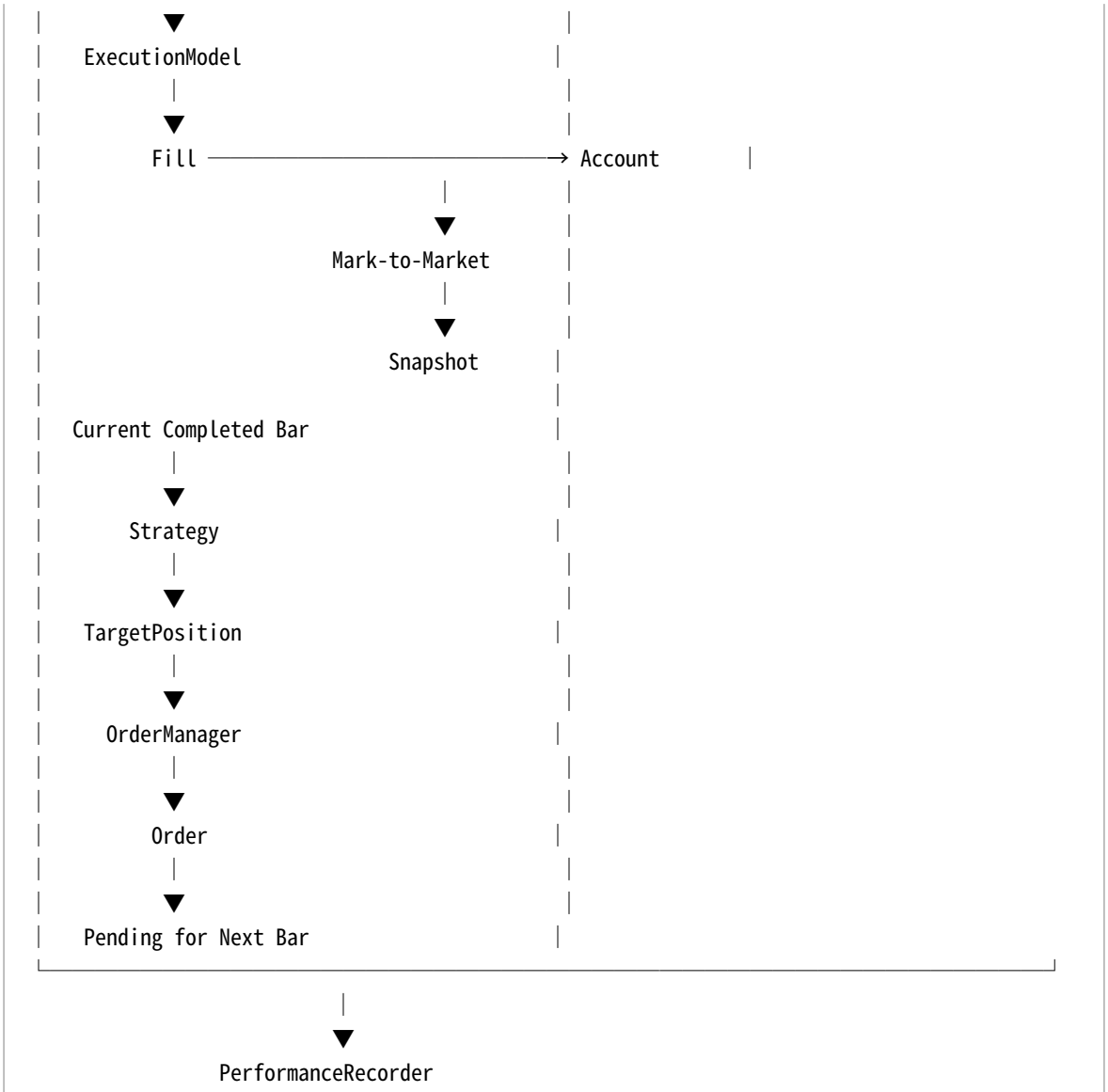
这就是判断V1架构是否成功的重要标准：

> **未来功能会增加，内部实现会改变，但核心交易概念不需要重新定义。**

五十二、最终推荐的V1系统骨架

最终可以压缩为：

```
                    Existing Data Engine
                           |
                           ▼
                     DataFacade
                           |
                           ▼
                    BarMarketFeed
                           |
                           ▼
                    BarMarketEvent
                           |
                           ▼
┌──────────────────────────────────────────────────────────────────────────┐
|                BacktestEngine                |                           |
|                                              |                           |
|   Previous Pending Orders                    |                           |
|           |                                   |                           |
```



五十三、V1最值得研究者亲自实现的部分

对于这一阶段，以下部分建议亲自设计和理解。

第一优先级：BacktestEngine事件顺序

一定亲自理解：

```
什么时候看到Bar
什么时候产生Target
什么时候生成Order
什么时候Fill
什么时候Account变化
```

第二优先级：TargetPosition → Order

理解：

> “我想持有什么”

与：

> “我要交易多少”

不是同一件事。

第三优先级：Order → Fill

理解：

> 交易意图

与：

> 已经发生的市场事实

不是同一件事。

第四优先级：Fill → Position → PnL

尤其亲自实现：

- 开仓；
- 加仓；
- 减仓；
- 平仓；
- 反向；
- 多空盈亏。

第五优先级：信息时点

亲自保证：

```
Signal Time
<
Order Execution Time
```

五十四、可以更多交给AI Agent的部分

在你定义清楚接口以后，可以较多委托：

- dataclass代码；
- ID生成；
- 配置类；
  -日志；
- Recorder；
- CSV/Parquet输出；
- Performance指标封装；
- pytest测试样板；
- CLI；
- 文档字符串；
- 类型注解；
- 工厂装配；
- 格式化和静态检查。

但：

> Engine事件顺序、Account状态转移、Position语义和时间因果测试，应由研究者本人审核。

五十五、V1开发顺序

最后给出实际编码顺序。

Step 1

定义核心数据模型：

```
BarMarketEvent
TargetPosition
Order
Fill
Position
AccountSnapshot
```

不要写复杂业务。

Step 2

写：

```
BarMarketFeed
```

把现有 `DataFacade.get_historical_vector()` 转成Bar事件流。

首先验证：

```
Adj Price
Raw Price
Dominant Contract
Timestamp
```

全部正确。

Step 3

写：

```
Account
```

先完全脱离Strategy。

手工调用：

```
apply_fill()
mark_to_market()
```

把：

- 开多；
- 平多；
- 开空；
- 平空；
- 反向；

全部测试通过。

Step 4

写：

```
ExecutionModel
CommissionModel
SlippageModel
```

只实现下一Bar开盘全部成交。

Step 5

写：

```
OrderManager
```

解决：

```
Target - Actual = Order
```

Step 6

写：

```
Strategy
```

实现最简单20 Bar趋势。

Step 7

写：

```
BacktestEngine
```

把所有模块串起来。

Step 8

写端到端黄金路径测试。

在此之前：

不开发Performance漂亮报表。

Step 9

最后增加：

```
PerformanceRecorder
```

输出：

- equity curve；
- positions；
- orders；
- fills；
- targets。

Step 10

真正运行历史数据。

此时第一次完成：

Data → Research → Decision → Trade → Account → Performance

闭环。

结论

V1 Bar级最小闭环的目的，并不是构造一个“小型但完整功能”的期货回测平台。

它真正要完成的是一次系统认知上的质变：

在V0中：

收益由研究公式计算出来。

而在V1中：

收益必须由一个按照时间顺序演化的交易账户产生。

因此，V1最重要的成果不是Sharpe ratio，也不是系统支持了多少策略，而是建立下面这组稳定交易语义：

Strategy表达判断；
TargetPosition表达希望持有什么；
Order表达准备如何交易；
Fill表达市场实际发生了什么；
Position表达账户现在持有什么；
Account表达这些交易如何改变资本状态；
Performance只忠实记录这一状态演化过程。

第一版的业务规则可以非常简单：

日频Bar；
固定一手；
下一Bar开盘成交；
全部成交；
固定滑点；
简化手续费；
简化账户。

但以下原则不能简化掉：

Signal不是Position；
Target不是Order；
Order不是Fill；
只有Fill能够改变实际Position；
连续价格用于研究，物理合约价格用于成交；
当前Bar产生的信息不能回到过去成交；
账户必须是资金、持仓和PnL的唯一权威来源。

如果这些原则从V1就稳定下来，那么未来：

```
V1 Bar最小闭环
        ↓
V2 完整期货账户
        ↓
V3 换月与物理执行
        ↓
V4 多品种组合与实验框架
        ↓
V5 Tick级微观执行
```

就不应表现为五次重新开发系统，而应该表现为：

同一个稳定交易骨架之上的业务语义不断丰富。

这就是V1真正应该达到的设计目标。